

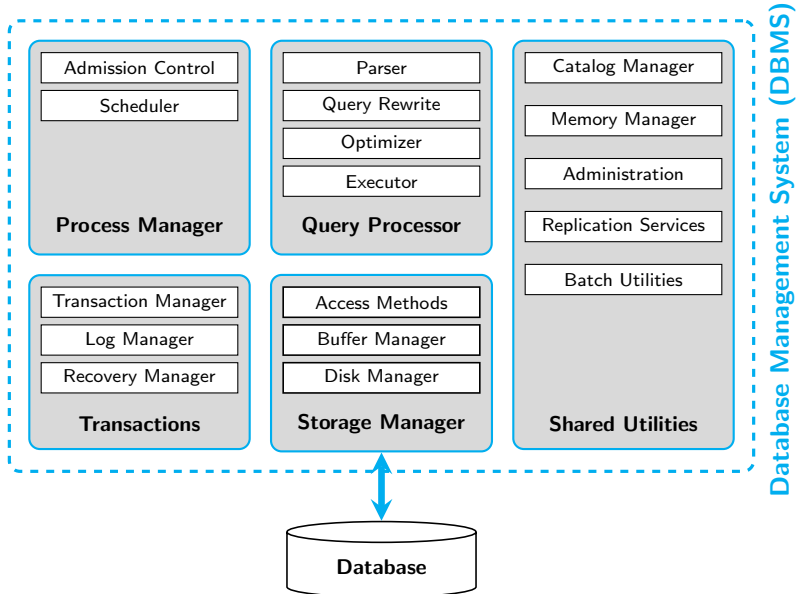
Almacenamiento y organización de los datos

Clase 12

IIC2413 - Sección 3

Prof. Cristian Riveros

Sistema de bases de datos por dentro



Outline

Almacenamiento de los datos

Organización de los datos

Buffer manager

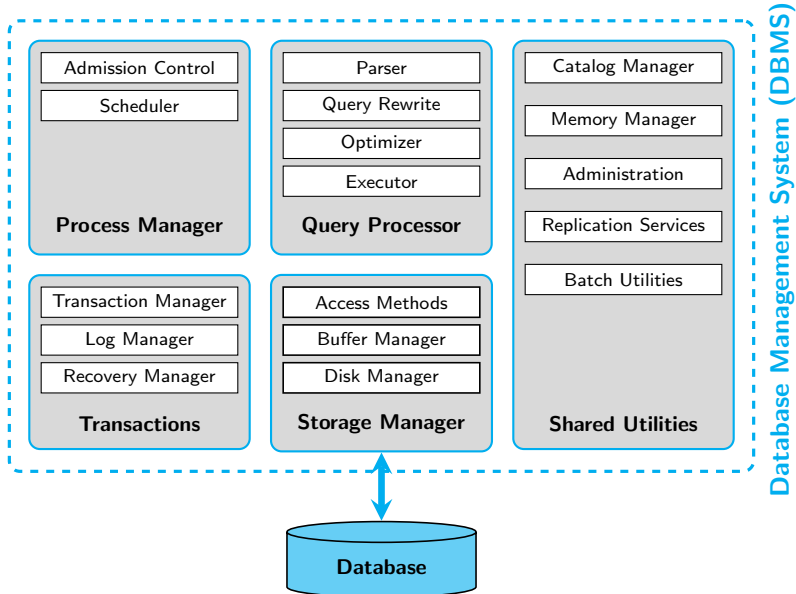
Outline

Almacenamiento de los datos

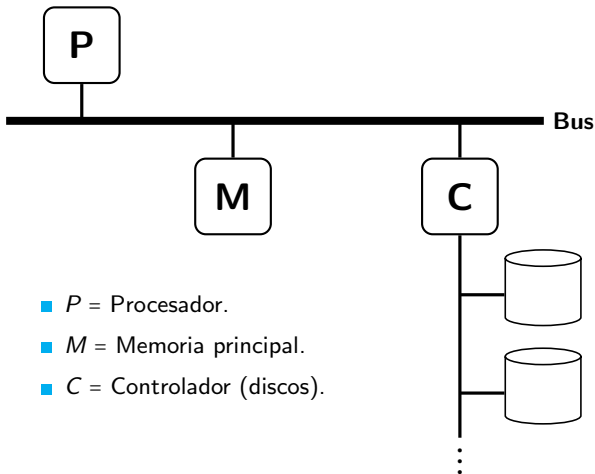
Organización de los datos

Buffer manager

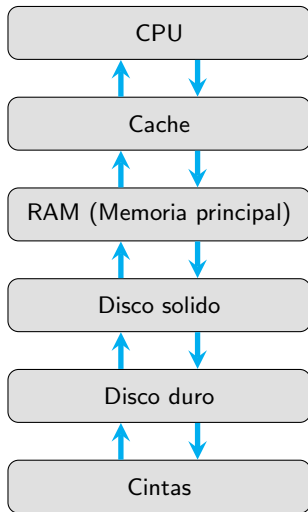
Almacenamiento de los datos



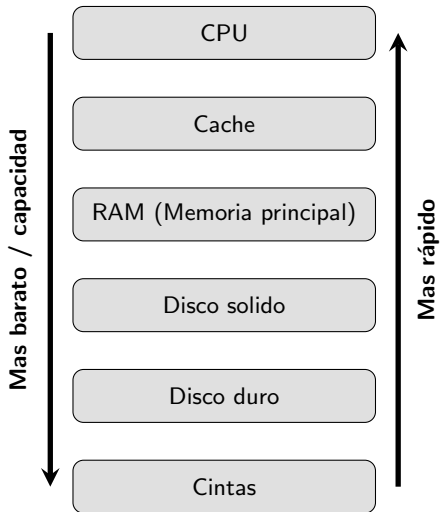
Modelo de computación y almacenamiento



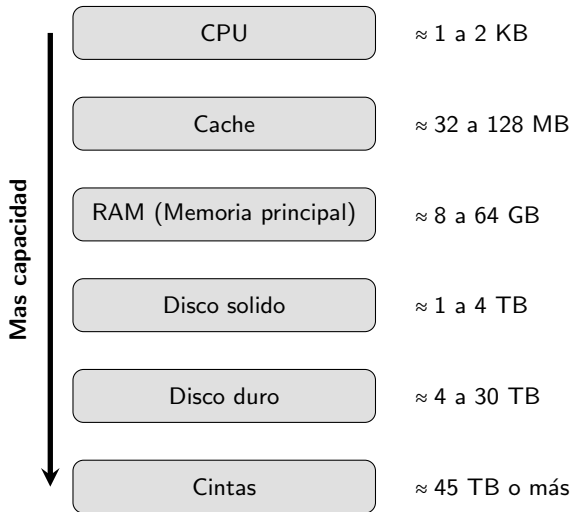
Distintos niveles de almacenamiento



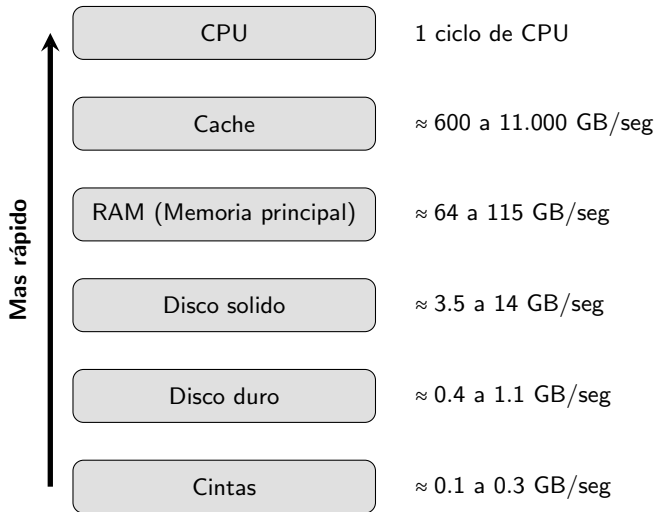
Distintos niveles de almacenamiento



Distintos niveles de almacenamiento



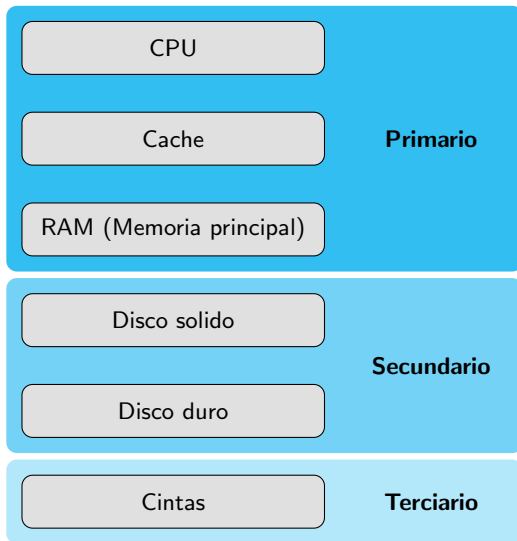
Distintos niveles de almacenamiento



Distintos niveles de almacenamiento



Distintos niveles de almacenamiento



Distintos niveles de almacenamiento

CPU

Cache

RAM (Memoria principal)

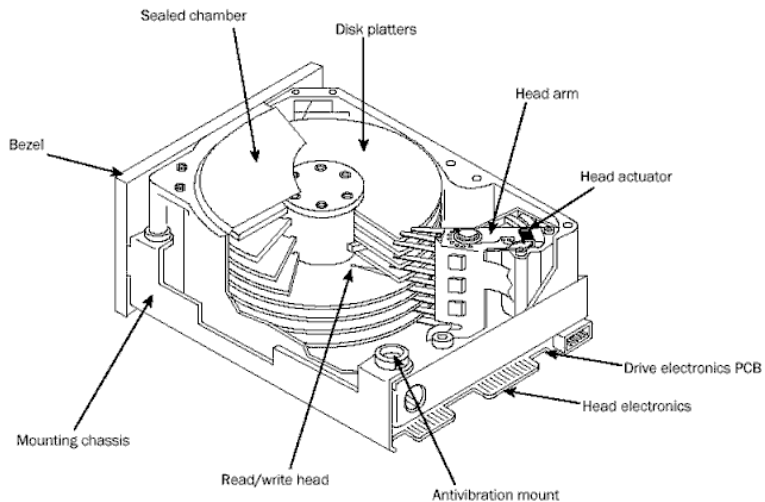
Disco solido

Disco duro

Cintas

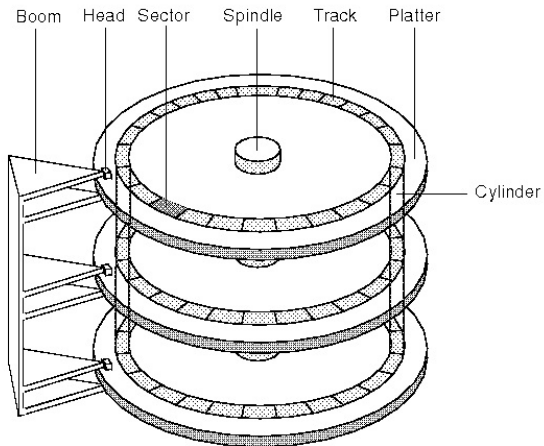
Importante
para un DBMS

Disco duro



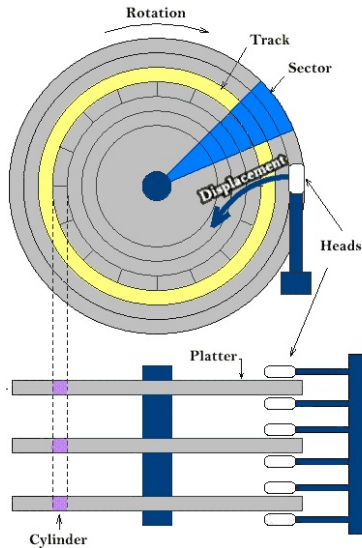
Componentes de un disco duro

- Sector
- Gap
- Track
- Cilindro
- Plato
- Cabeza lectora



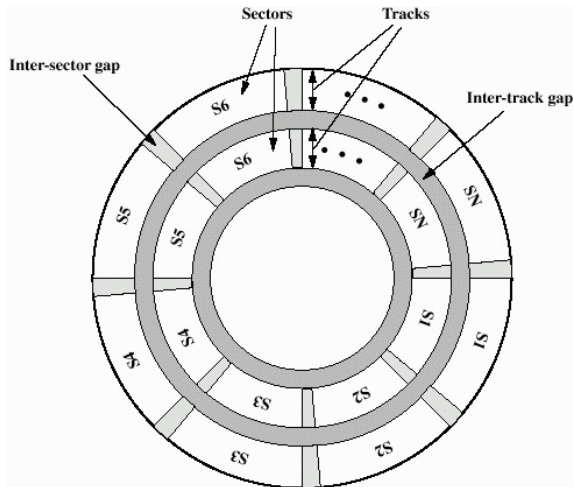
Componentes de un disco duro

- Sector
- Gap
- Track
- Cilindro
- Plato
- Cabeza lectora



Componentes de un disco duro

- Sector
- Gap
- Track
- Cilindro
- Plato
- Cabeza lectora



Sectores vs Blocks vs Pages en un disco

Definición

- **Sector:** unidad **física** minima de almacenamiento (**Disco**).
- **Block:** unidad **lógica** minima de almacenamiento (**Disco**).
- **Page:** unidad **lógica** minima de almacenamiento (**Sistema**).

Un sistema de bases de datos razonan/optimizan **en páginas**

Demora en la búsqueda de un bloque

$$\begin{aligned} \text{Tiempo total} &= \text{Tiempo de busqueda} && (\text{Seek time}) \\ &+ \\ &\text{Retraso rotacional} && (\text{Rotational delay}) \\ &+ \\ &\text{Velocidad de transferencia} && (\text{Transfer rate}) \end{aligned}$$

Definición

- **Tiempo de busqueda:** tiempo en posicionar cabeza en el track.
- **Retraso rotacional:** tiempo en rotar el disco hasta el primer sector.
- **Vel. de transferencia:** tiempo en leer los sectores (incl. gaps).

Demora en la búsqueda de un bloque

$$\begin{aligned} \text{Tiempo total} &= \text{Tiempo de busqueda} && (\text{Seek time}) \\ &+ \\ &\text{Retraso rotacional} && (\text{Rotational delay}) \\ &+ \\ &\text{Velocidad de transferencia} && (\text{Transfer rate}) \end{aligned}$$

Definición

- **Tiempo de busqueda:** 3 ms $\rightarrow$ 15 ms (promedio).
- **Retraso rotacional:** 2 ms $\rightarrow$ 7 ms (promedio).
- **Vel. de transferencia:** $\leadsto$ 0 (dependiendo del tamaño de los datos)

Demora en la búsqueda de un bloque

Ejemplo: Disco duro 2008

- **Rotación:** 7200 rpm (8.33 ms por vuelta)

- **Movimiento brazo:**

1 ms para empezar/parar brazo

1 ms por cada 4000 tracks

Numero de tracks: 65.536 tracks

- **Transferencia de datos:** 0.13 ms por bloque

	Min	Max	Promedio
Seek	0 ms	17.38 ms	6.46 ms
Rotational	0 ms	8.33 ms	4.17 ms
Transfer	0.13 ms	0.13 ms	0.13 ms
Total	0.13 ms	25.84 ms	10.76 ms

¿cuánto es la demora si debo buscar **1000 bloques distintos**?

Demora en la búsqueda de un bloque

Disco de estado sólido

- No hay costo ni de Seek ni de Rotación.
- Acceso aleatorio toma ~ 0.1 ms.
- Aún así es 100 veces más lento que una RAM.

Moraleja

1. Tiempos de búsqueda en un disco pueden ser significativos.
2. Datos relacionados deben ser almacenados en bloques contiguos.
3. Minimizar número de accesos a disco.

Modelo de computación basado en I/O

Hechos / suposiciones

- Espacio del disco duro >> espacio en RAM.
- Tiempo de acceso a disco >> tiempo de acceso RAM.

Dominación del costo I/O:

*“El tiempo para acceder al disco es mucho mayor que el tiempo necesario para manejar datos en memoria. Entonces, el **número de accesos a disco** (Disk I/O) es una buena aproximación al tiempo del algoritmo y, por lo tanto, debe ser minimizado.”*

Garcia-Molina, Ullman, & Widom, 2008

El costo en BD es medido según los **accesos a páginas**

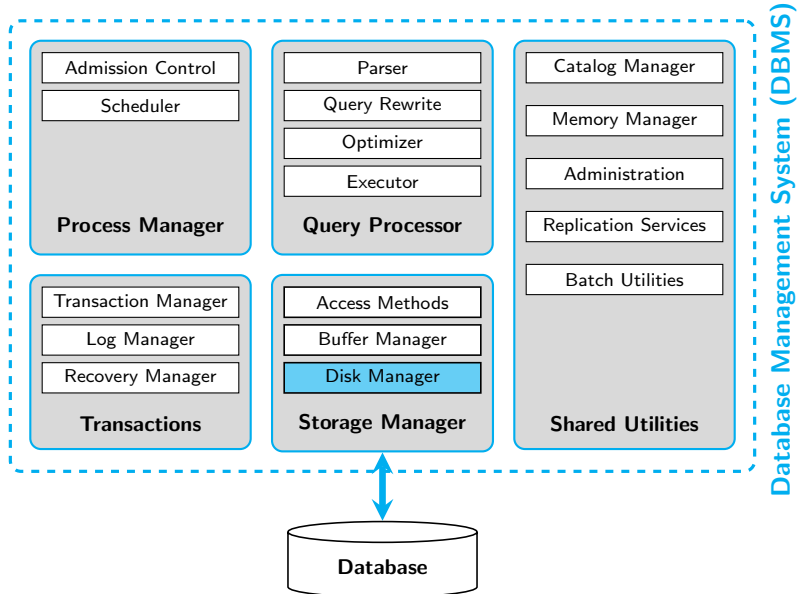
Outline

Almacenamiento de los datos

Organización de los datos

Buffer manager

Organización de los datos



Organización basado en páginas

1 **página** $\approx$ 1 **bloque** en disco

Organización de una relación

Cada **relación** es representada como un conjunto de páginas y cada **página** contienen un subconjunto de las tuplas.

¿cuáles son las **ventajas** de organizar una relación en páginas?

- Manejo granular del contenido en disco.
- Optimización del acceso al disco.
- Facilidad para el manejo de transacciones.

Organización basado en páginas

1 **página** $\approx$ 1 **bloque** en disco

Organización de una relación

Cada **relación** es representada como un conjunto de páginas y cada **página** contienen un subconjunto de las tuplas.

Preguntas

1. ¿cómo almacenamos una tupla?
2. ¿cómo almacenamos varias tuplas en una página?
3. ¿cómo almacenamos una relación en varias páginas?

¿cómo almacenamos una tupla?

Primera solución: secuencia de bytes + separadores

Players

id	name	birth	team
10	Lionel Messi	24/6/1987	Inter Miami
9	Erling Haaland	21/7/2000	Manchester City

#	10	\$	Lionel Messi	\$	24/6/1987	\$	Inter Miami	#
---	----	----	--------------	----	-----------	----	-------------	---

¿cómo representamos un valor null?

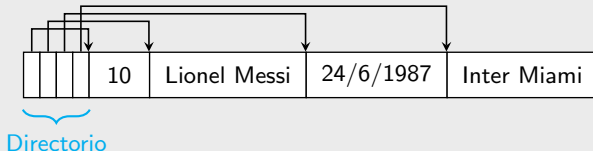
¿cuáles son las **ventajas/desventajas?**

¿cómo almacenamos una tupla?

Segunda solución: directorio

Players

id	name	birth	team
10	Lionel Messi	24/6/1987	Inter Miami
9	Erling Haaland	21/7/2000	Manchester City



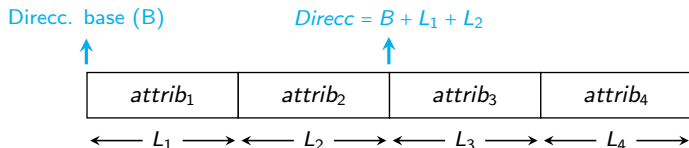
¿cómo representamos un valor null?

¿cuáles son las **ventajas/desventajas**?

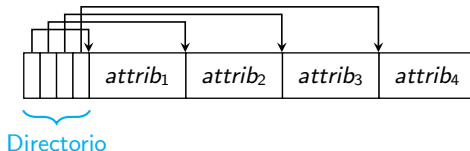
¿cómo almacenamos una tupla?

Solución final

Atributos de **tamaño fijo**:



Atributos de **tamaño variable**:



Organización basado en páginas

1 **página** $\approx$ 1 **bloque** en disco

Organización de una relación

Cada **relación** es representada como un conjunto de páginas y cada **página** contienen un subconjunto de las tuplas.

Preguntas

1. ¿cómo almacenamos una tupla?
2. ¿cómo almacenamos varias tuplas en una página?
3. ¿cómo almacenamos una relación en varias páginas?



¿cómo almacenamos varias tuplas en una página?

Record ID

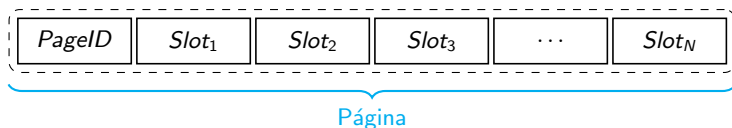
- Identificador único de cada tupla.
- Necesario para mantener unicidad de cada tupla.

¿cuál es una **buena elección de RID**?

$RID = (PageID, \text{posición en la página})$

¿cómo almacenamos varias tuplas en una página?

Tuplas de tamaño fijo



Ejemplo

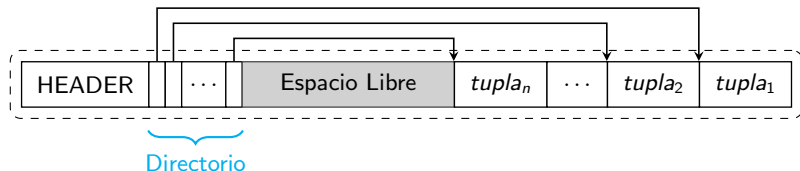


$RID = (PageID, slot)$

¿cómo **insertamos/eliminamos** una tupla?

¿cómo almacenamos varias tuplas en una página?

Tuplas de tamaño variable



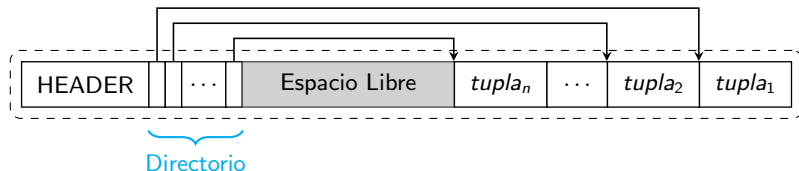
HEADER = *PageID* + número de tuplas + tamaño directorio

RID = (*PageID*, *DirectoryID*)

¿cómo **insertamos/eliminamos** una tupla?

¿cómo almacenamos varias tuplas en una página?

Tuplas de tamaño variable

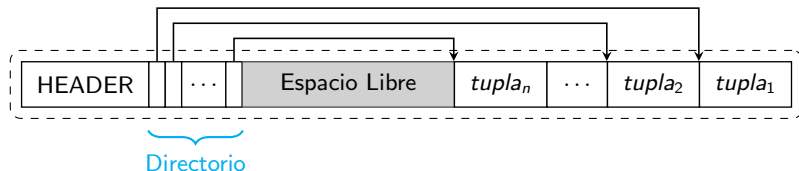


¿Ventajas?

- Manejo de tuplas de tamaño variable.
- Eliminación de tuplas.
- Mover tuplas dentro de una página (indireccionamiento), o dentro de distintas páginas (forwarding address).

¿cómo almacenamos varias tuplas en una página?

Tuplas de tamaño variable



*"Tuplas de tamaño variable
pero **no mayor** al tamaño de una página."*

¿cómo almacenamos tuplas/atributos de **tamaño mayor a una página?**

char(n) vs. varchar(n) vs. clob(n)

BLOB = Binary Large Object

CLOB = Character Large Object

- Soportado por la mayoría de DBMS modernos.
- Incluyen: imágenes, videos, textos, etc.

¿cuál es la **diferencia** entre char(n), varchar(n), y clob(n)?

Organización basado en páginas

1 **página** $\approx$ 1 **bloque** en disco

Organización de una relación

Cada **relación** es representada como un conjunto de páginas y cada **página** contienen un subconjunto de las tuplas.

Preguntas

1. ¿cómo almacenamos una tupla?
2. ¿cómo almacenamos varias tuplas en una página?
3. ¿cómo almacenamos una relación en varias páginas?



¿cómo almacenamos una relación en varias páginas?

Solución 1: páginas consecutivas

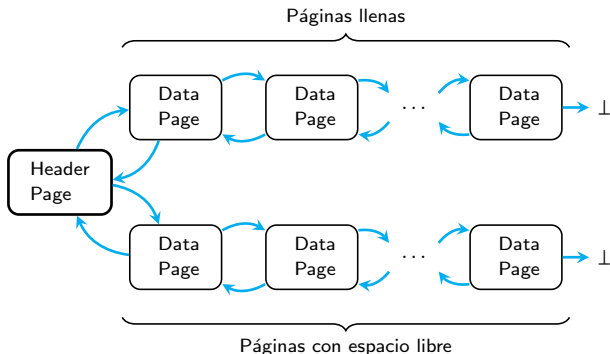


¿qué **problemas** tiene esta solución?

- Requiere un espacio contiguo en disco.
- Fragmentación de espacio en las páginas.
- Buscar en todas las páginas para encontrar espacio.

¿cómo almacenamos una relación en varias páginas?

Solución 2: listas ligadas

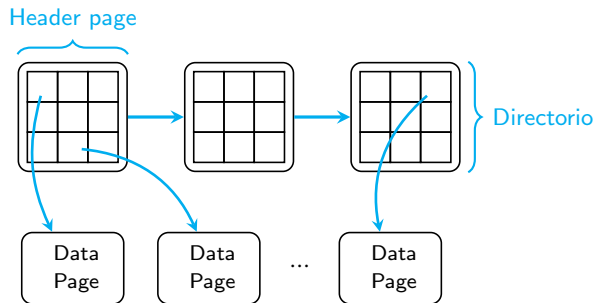


¿qué **problemas** tiene esta solución?

- Buscar en todas las páginas vacías por espacio para una tupla.

¿cómo almacenamos una relación en varias páginas?

Solución 3: Directorio de páginas



¿qué **ventajas** tiene esta solución?

- Directorio contiene meta-información sobre las páginas (espacio libre).
- Mayor eficiencia en la búsqueda de espacio libre.

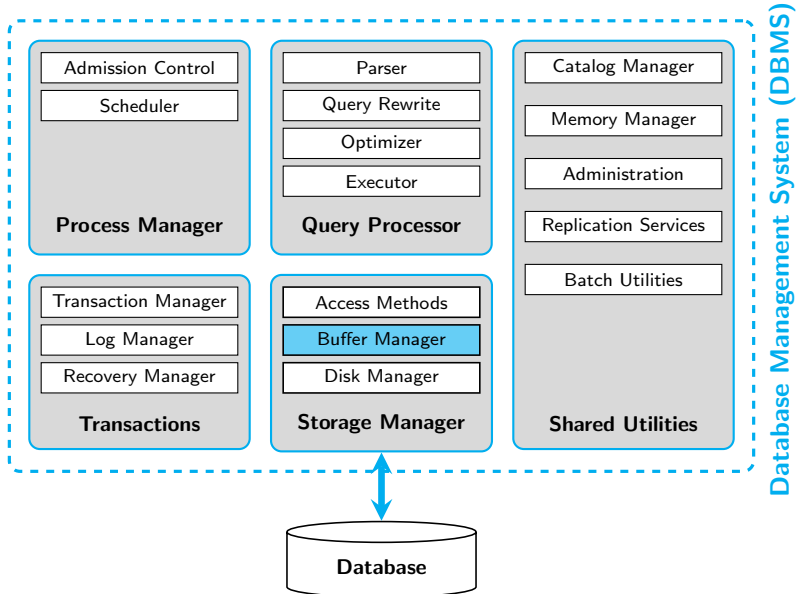
Outline

Almacenamiento de los datos

Organización de los datos

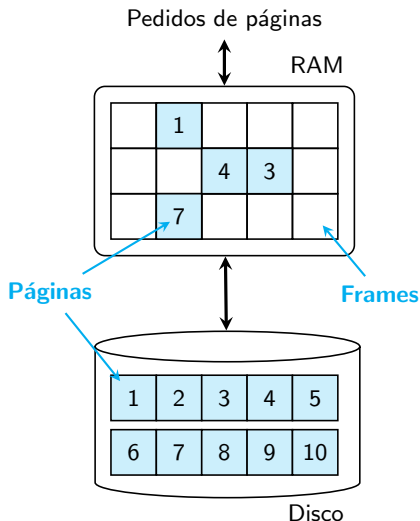
Buffer manager

Buffer manager



Buffer manager

- Mediador entre el disco y la memoria principal.
- Cuenta con una cantidad restringida de memoria RAM.
- Páginas son traídas a memoria a pedido.
- Encargado de decir que páginas eliminar cuando el buffer esta lleno.



Buffer manager

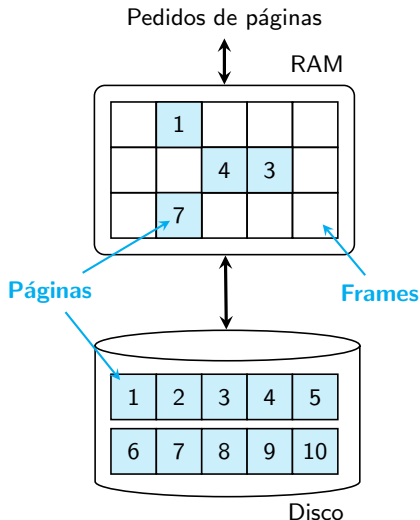
Cada **frame** tiene dos variable:

1. **#pin** = cantidad de procesos que estan usando esta página.
2. **Dirty** = si el contenido es necesario guardarlo en memoria.

Procesos acceden al Buffer manager usando las funciones:

```
pin(pageno)
```

```
unpin(pageno, dirty)
```



Interfaz para acceder el buffer manager

`pin( pageno )`

1. Solicita la página `pageno` al buffer manager.
2. Si la página `pageno` no esta en memoria:
 - Se selecciona un frame vacío `X`.
 - Se trae la página `pageno` a memoria y se carga en `X`.
 - Se actualiza `X.#pin := 1` y `X.dirty := false`.
3. Si la página `pageno` esta en memoria, se incrementa su `#pin` en 1.
4. Buffer manager retonar una referencia al frame que contiene `pageno`.

Problemas

- ¿qué ocurre si no hay un frame vacío?
- ¿qué ocurre si todos los frames tienen `#pin` mayor que 1?

Interfaz para acceder el buffer manager

`unpin( pageno, dirty )`

1. Solicita la liberación de la página `pageno` (almacenada en el frame X).
2. Se decrementa el $X.\#pin$ en uno.
3. Se actualiza $X.dirty := true$ si la página fue modificada.

Problemas

- ¿cuándo son guardados los datos de la página en disco?
- ¿qué ocurre si un proceso nunca hace `unpin` de su página?

Interfaz para acceder el buffer manager

Requisitos de la interfaz pin y unpin

1. Cada proceso (de una BD) debe mantener las funciones pin y unpin **correctamente anidadas**.

```
 $d \leftarrow \text{pin}(p);$   
 $\vdots$   
(proceso lee y usa los datos en la dirección de memoria  $d$ )  
 $\vdots$   
 $\text{unpin}(p, \text{false});$ 
```

2. Cada proceso (de una BD) debe liberar una página lo antes posible.

Políticas de reemplazo

“La efectividad del buffer manager depende directamente de la política de reemplazo usada.”

Diferentes tipos de políticas / heurísticas

- FIFO
- Least Recently Used (LRU)
- Random

Estas son heurísticas y todas tienen un **peor caso**